

classmate

Date _____

Page _____

Problem Solving

Problem solving is an agent based system that finds sequence of actions that lead to desirable states from the initial state.

- Problem solving agent is a kind of goal based agent which always tries to achieve goals.
- Searching is the most commonly used technique of problem solving in AI.

Four steps of problem solving are :

1. Goal Formulation:

A goal is a state that the agent is trying to reach. This step formulates the goal based on current situation and the agent's performance.

2. Problem Formulation:

This is the process of deciding what actions and states to consider, given goal.

- Problem formulation must follow goal formulation.

3. Search method:

Determine the possible sequence of actions that leads to the states of known values and then choosing the best sequence.

4. Execute:

Once the search algorithm returns a solution to the problem, the solution is then executed by the agent.

classmate

Date _____

Page _____

A **problem** can be defined formally by 4 components:

1. Initial state:

It is the state from which agent start solving the problem.

2. State description / successor function:

A description of the possible actions available to the agent.

3. Goal test:

Determine whether the current state is goal state or not.

4. Path cost:

sum of cost of each path from initial state to the given state.

A **solution to a problem** is path (sequence of actions) from the initial state to a goal state. Optimal solution has the lowest path cost.

E.g.

8-puzzle problem

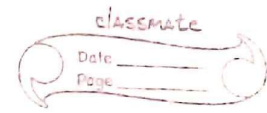
7	3	
6	8	4
5	2	1

Start state

	1	2
3	4	5
6	7	8

Goal state

Problem formulation of 8-puzzle problem is given by:



1. States:

Description of the eight tiles and location of the blank tile.

2. Goal test:

checks whether the state matches the goal configuration.

3. Successor function / Actions:

Generate the legal states from trying the four actions { left, right, top, down }.

4. Path cost:

Each step cost 1.

Q. Why problem formulation must follow goal formulation?

Solⁿ

In goal formulation, we decide which aspects of the world we are interested in and which aspects can be ignored. In goal formulation process, the goal is to be set and we should assess those states in which the goal is satisfied. In problem formulation, we decide how to manipulate the important aspects and ignore the others. So without doing goal formulation, if we do the problem formulation, we would not know what to include in our problem & what to leave and what should be achieved. So problem formulation must follow goal formulation.

classmate

Date _____

Page _____

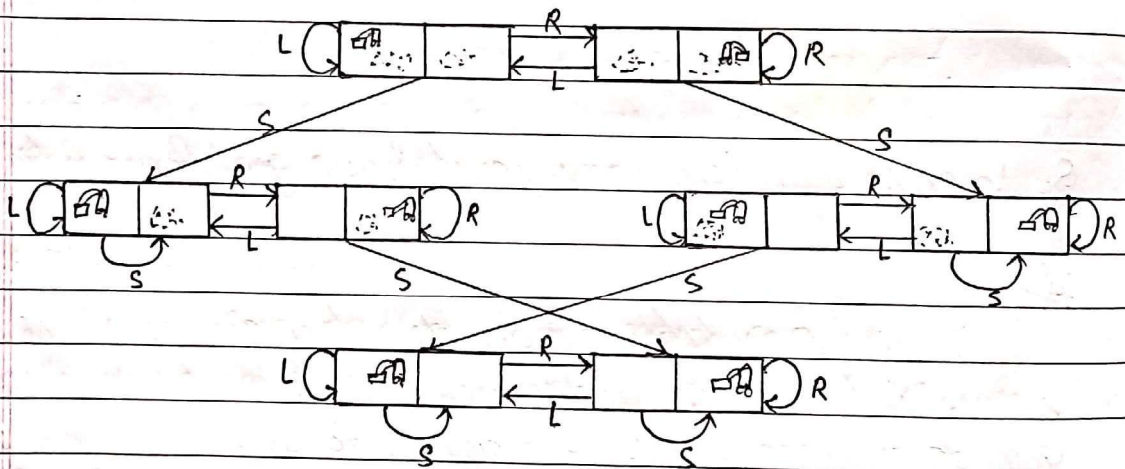
State space Representation

The state space is commonly defined as a directed graph in which each node is a state and each arc represents the application of an operator transforming a state to a successor state.

A solution is a path from the initial state to a goal state.

- state space is the set of all states reachable from the initial states.

state space representation of vacuum world problem:



States:

Two locations with or without dirt = 8 states

Initial state:

Any state can be initial

Actions:

{ Left, Right, suck }

Goal test:

check whether squares are clean.

Path cost:

number of actions to reach goal.

Searching

→ Searching is the process of finding the required states or nodes.

search problem

→ is to find path from initial state to goal state.

A search problem consists of:

- A state space
- A start state
- A goal state

* Types of search method

1. Uniformed (Blind) search method:

This type of search does not use any domain knowledge. This means it does not use any information to judge where the solution is likely to lie. Uniformed search methods use only the information available in the problem definition.

E.g.

- Breadth first search
- Depth first search
- Uniform cost search
- Depth limited search
- Iterative deepening search
- Bidirectional search

2. Informed (Heuristic) search method:

Heuristic search uses domain-dependent (heuristic) information in order to search the space more efficiently. It uses a heuristic function $h(n)$ that estimates how close we are to a goal. This function is used to estimate the cost from a state n to the closest goal.

E.g.

- Greedy search
- A* search
- Hill climbing search
- simulated annealing etc.

* Performance measures:

We will evaluate the performance of a search algorithm in four ways:

1. Completeness:

Whether the search is complete or not.

An algorithm is said to be complete if it definitely finds solution to the problem, if exist.

2. Time complexity:

Time required to get search solution. usually measured in terms of the number of nodes expanded.

3. space complexity:

Maximum number of nodes to be visited (stored in memory).

4. Optimality:

If a solution is found, is it guaranteed to be an optimal one? For e.g. is it the one with minimum cost?

$d \rightarrow$ depth of solⁿ

$m \rightarrow$ max^m depth of search space

$b \rightarrow$ branching factor

$\rightarrow$ max^m no. of nodes (children) generated by parent node.

} Time & space complexity are measured in terms of d, m & b .

classmate

Date _____

Page _____

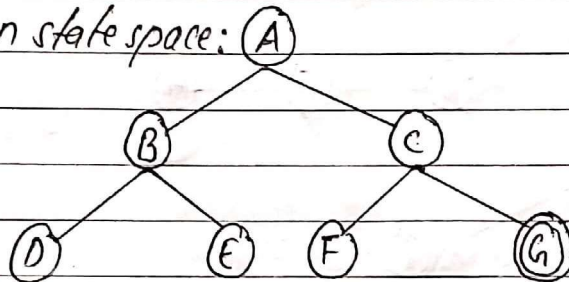
Breadth first search (BFS)

→ nodes having lowest depth.

It expands the shallowest unexpanded node first. Starting from the root node (initial state) explores all children of the root node, left to right. If no solution is found, expands the first (leftmost) child of the root node, then expands the second node at depth 1 and so on until a solution is found.

e.g.

① Given state space: (A)

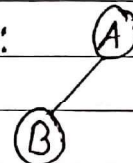


start state = A, Goal state = G

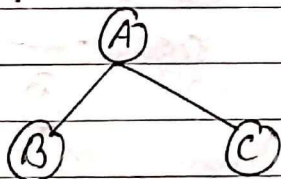
step 1:



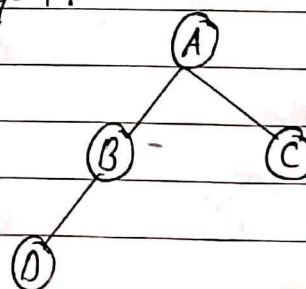
step 2:

} fringe
collection of nodes.

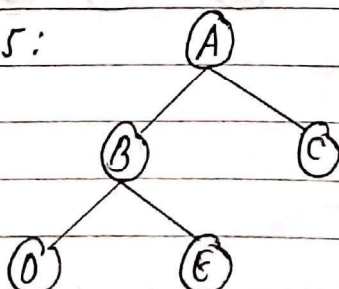
step 3:



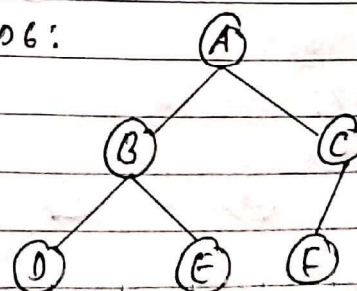
step 4:



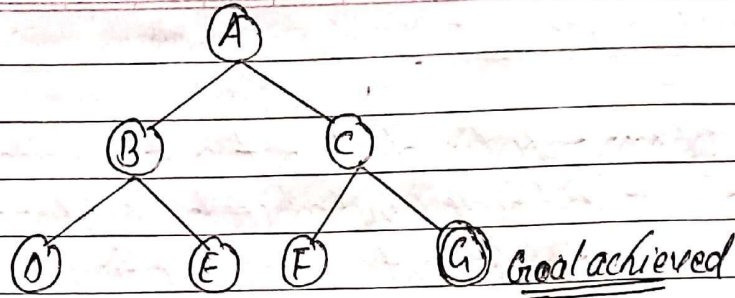
step 5:



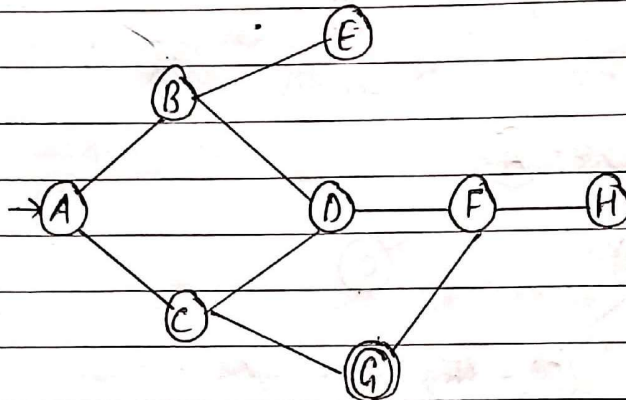
step 6:



step 7:



2

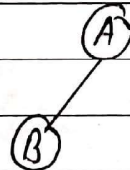


let A be the start state and G be the goal state.

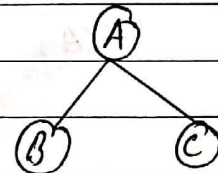
step 1:



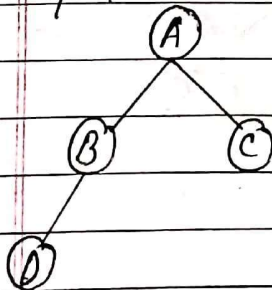
step 2:



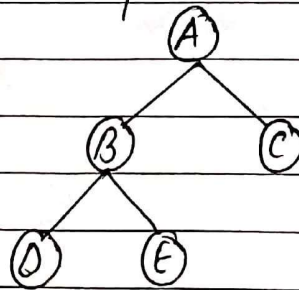
step 3:



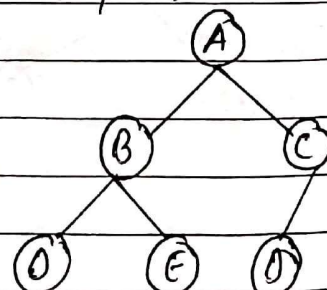
step 4:



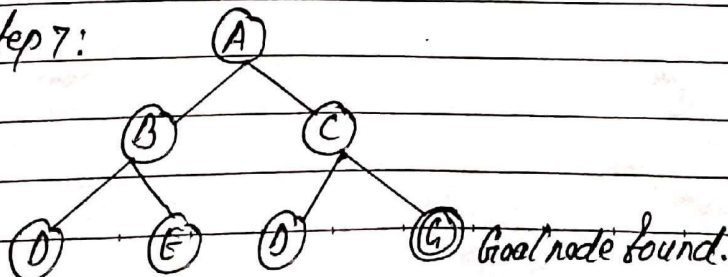
step 5:



step 6:



step 7:



classmate

Date _____

Page _____

✓ **BFS is known as complete.**

↳ It guaranteed to get solution if infinite nodes (depth) with finite final nodes.

BFS evaluation/properties

1. Completeness:

Complete if the goal node is at finite depth.

2. Optimality:

It is guaranteed to find the shortest path.

3. Time complexity:

No. of nodes to be visited.

For branching factor b and depth level d .

$$b + b^2 + b^3 + \dots + b^d + (b^{d+1} - b) = O(b^{d+1})$$

4. Space complexity:

↳ it stores the entire fringe.

$$\begin{aligned} \text{Total no. of nodes in memory} &= 1 + b + b^2 + \dots + b^d + (b^{d+1} - b) \\ &= O(b^{d+1}) \end{aligned}$$

Advantages

- If any solution exists then BFS guarantees to find it.
- If there are many solutions, BFS will always find the shortest path solution.

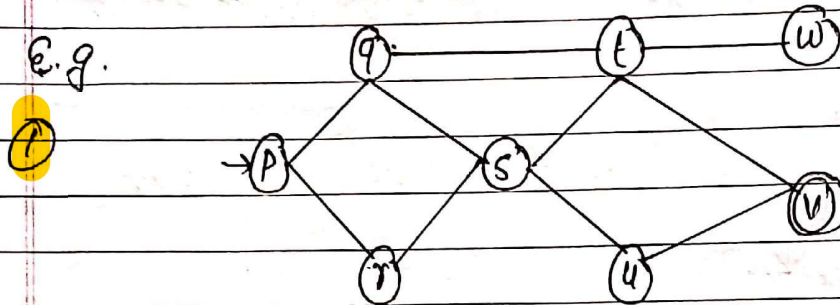
Disadvantages

- All nodes are to be generated at any level. So even unwanted nodes are to be remembered. Memory wastage.
- High time

Depth First search (DFS)

It expands the deepest unexpanded node first.
 → It expands the root node, then the leftmost child of the root node, then the leftmost child of that node and so on, only when the search hits dead end does the search backtrack and expands nodes at higher levels.

e.g.



let start state = P and goal state = V

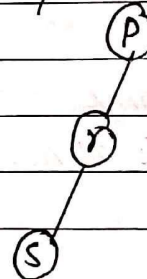
step 1:



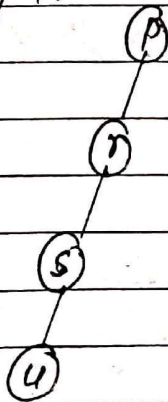
step 2:



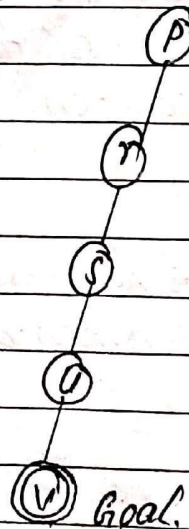
step 3:



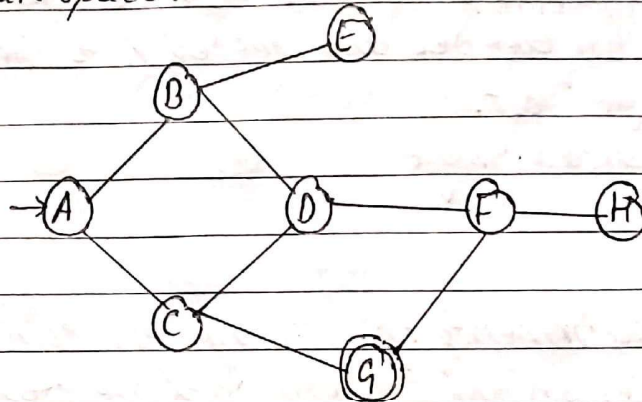
step 4:



step 5:



② Given state space:

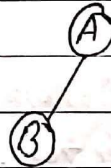


start state = A, goal state = G

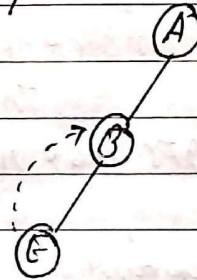
step 1:



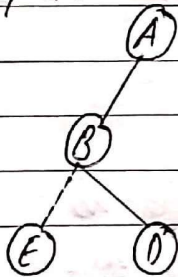
step 2:



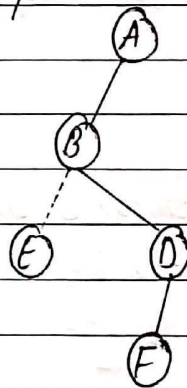
step 3:



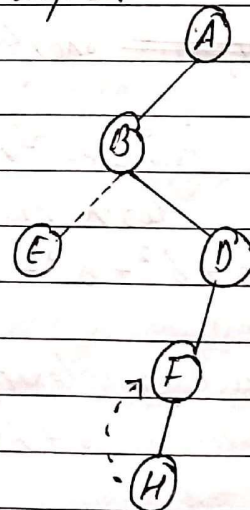
step 4:



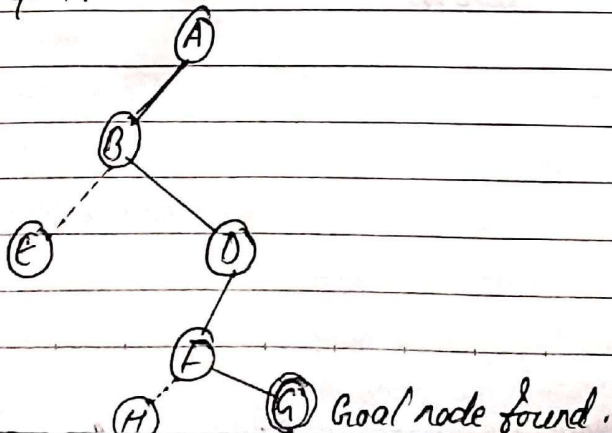
step 5:



step 6:



step 7:



↳ Disadvantage.

→ Memory requirements in DFS are less compared to BFS as only nodes on the current path are stored.

1. Completeness:

Incomplete if there is infinite structure then DFS may not find solution.

The first solution found by the DFS may not be ~~complete~~ shortest.

with branching factor b and depth as m

$$b + b^2 + b^3 + \dots + b^m = O(b^m) \quad m \geq d$$
$$1 + b + b + \dots + b = O(bm)$$

Depth limited search

In order to avoid the infinite loop condition arising in DFS, in depth limited search technique, depth-first search is carried out with a predetermined depth limit L .

If $L \geq d$, it gives solution

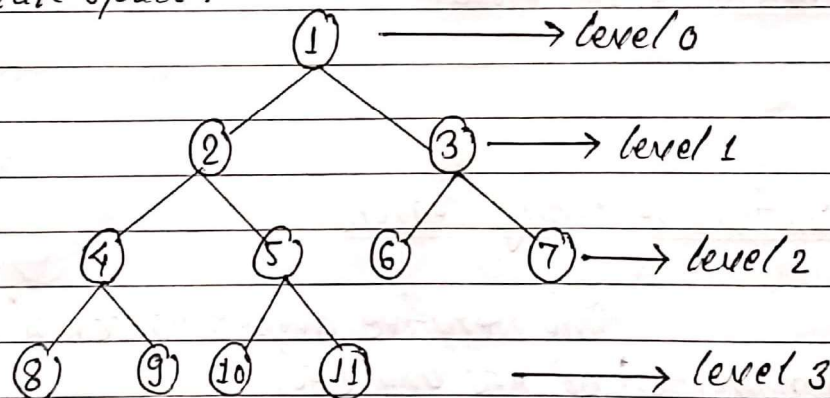
If $L < d$, it will be incomplete i.e. there is no solⁿ on the given depth limit.

where, d is the depth of the solution.

- The level of each node needs to be calculated to check whether it is within the specified depth limit.

e.g.

Given state space:



let depth limit = 2 and goal node = 11.

step 1:

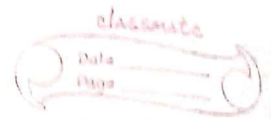


step 2:

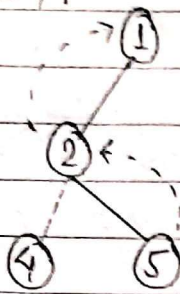


step 3:

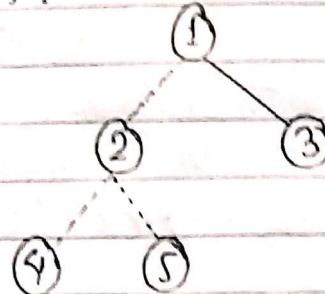




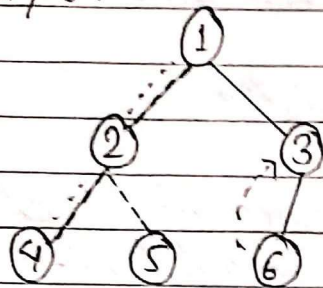
step 4:



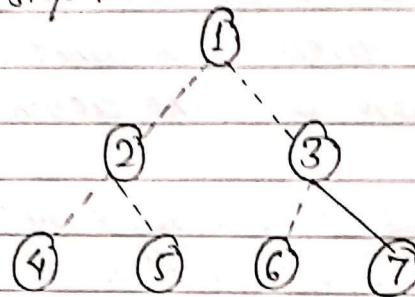
step 5:



step 6:



step 7:



Since depth limit we taken is 2 but goal node at level 3 so search is unsuccessful.

Iterative Deepening search

The iterative deepening search algorithm is a combination of DFS and BFS.

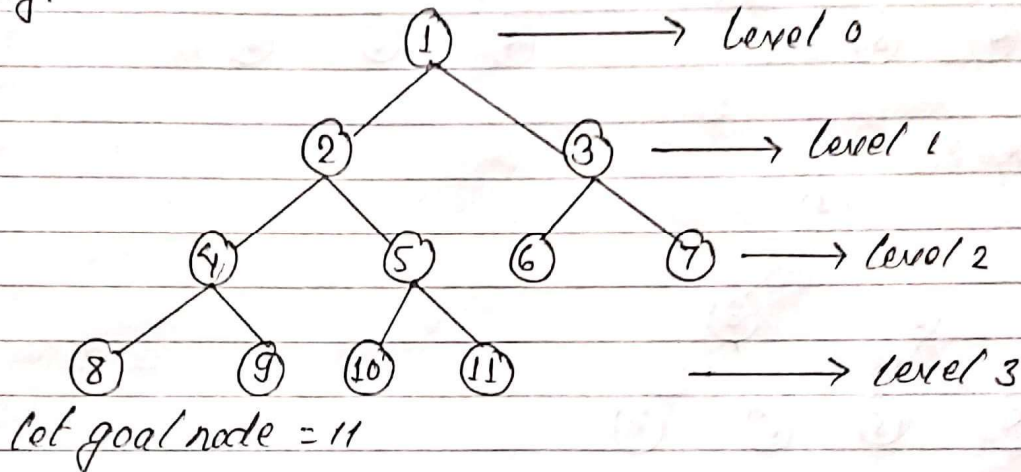
In this algorithm, starting at depth limit $d=0$, we iteratively increase the depth limit performing a depth limited search for each depth limit until the goal is found.

Implementation:

1. Initialize depth limit zero.
2. Repeat until the goal node is found.
 - a. call depth limited search with new depth limit.
 - b. Increment depth limit to next level.

→ It combines the advantages of both BFS & DFS, taking completeness and optimality of BFS and the modest memory requirements of DFS.

e.g.



limit = 0

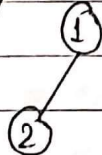


limit = 1

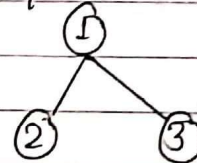
step 1:



step 2:



step 3:

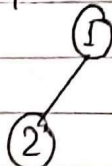


limit = 2

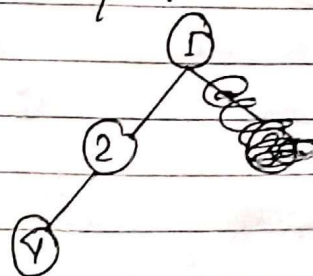
step 1:



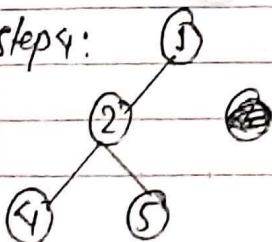
step 2:



step 3:



step 4:

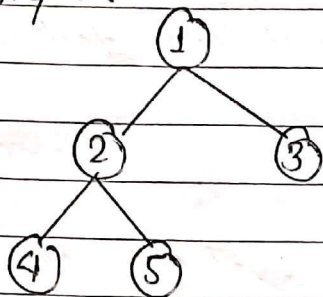


classmate

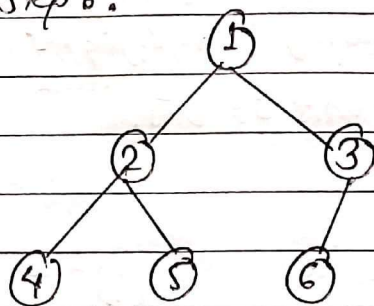
Date: _____

Page: _____

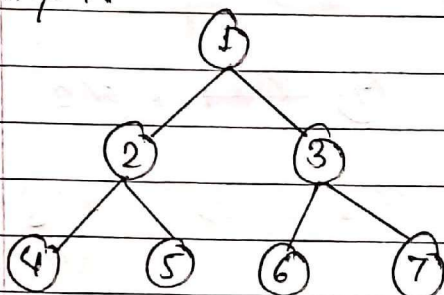
step 5:



step 6:



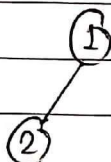
step 7:

limit = 3

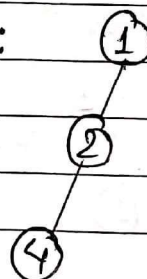
step 1:



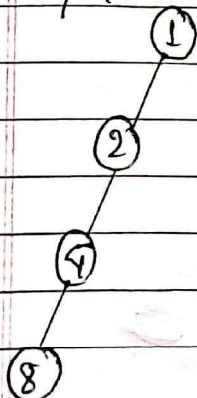
step 2:



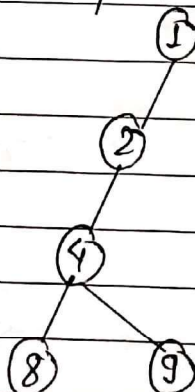
step 3:



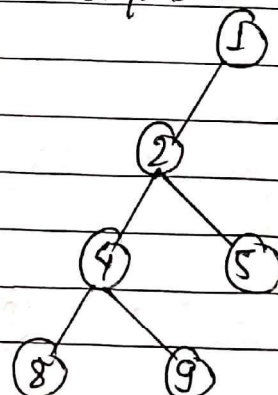
step 4:



step 5:



step 6:

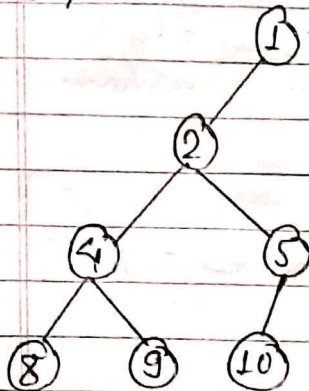


classmate

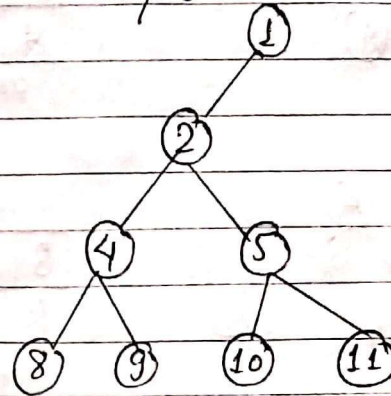
Date _____

Page _____

step 7:



step 8:



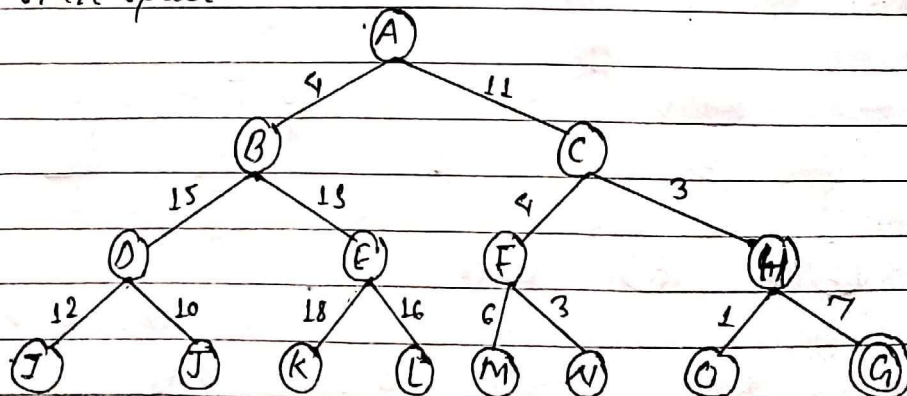
Goal node found.

Uniform cost search

- Searching algorithm used for weighted state space.
- Priority queue for storing nodes in state space is maintained, where least cost paths are given higher priority.
- Node at head of the queue is expanded first.
- The queue is updated at each expansion of nodes. (deletion of node visited & insertion of node to be visited).

E.g.

① Given state space



let start state = A & Goal state = G

Step 1:

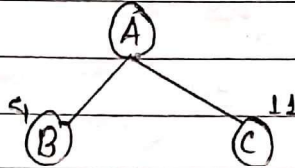
(A)⁰

A	
---	--

queue

visited = {A}

Step 2:

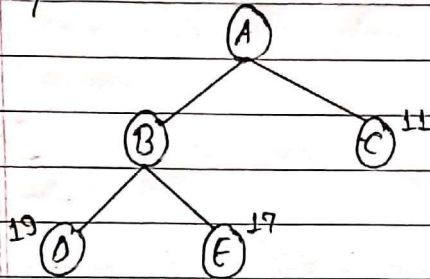


B	C	
---	---	--

↑

visited = {A, B}

Step 3:

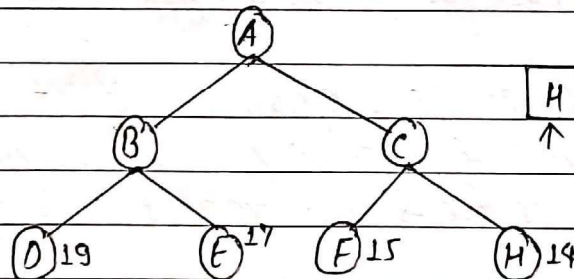


C	E	D
---	---	---

↑

visited = {A, B, C}

Step 4:

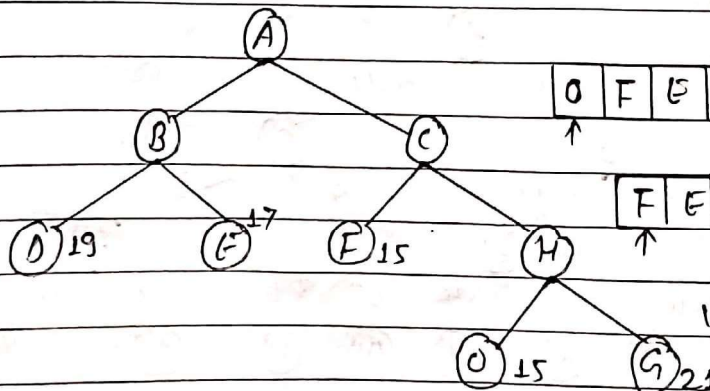


H	F	E	D
---	---	---	---

↑

visited = {A, B, C, H}

Step 5:



I	F	E	D	G
---	---	---	---	---

↑

F	E	D	G
---	---	---	---

↑

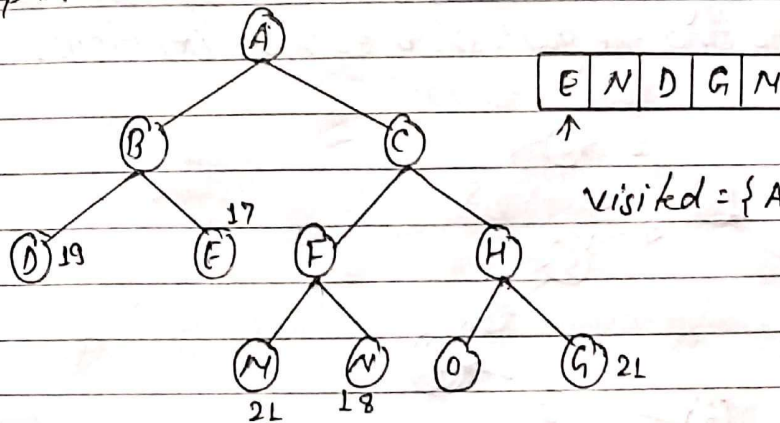
visited = {A, B, C, H, D, F}

classmate

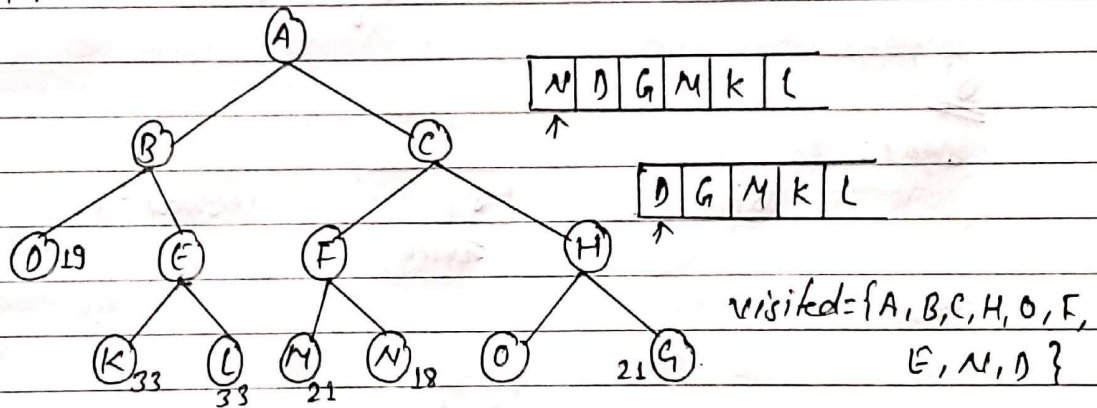
Date _____

Page _____

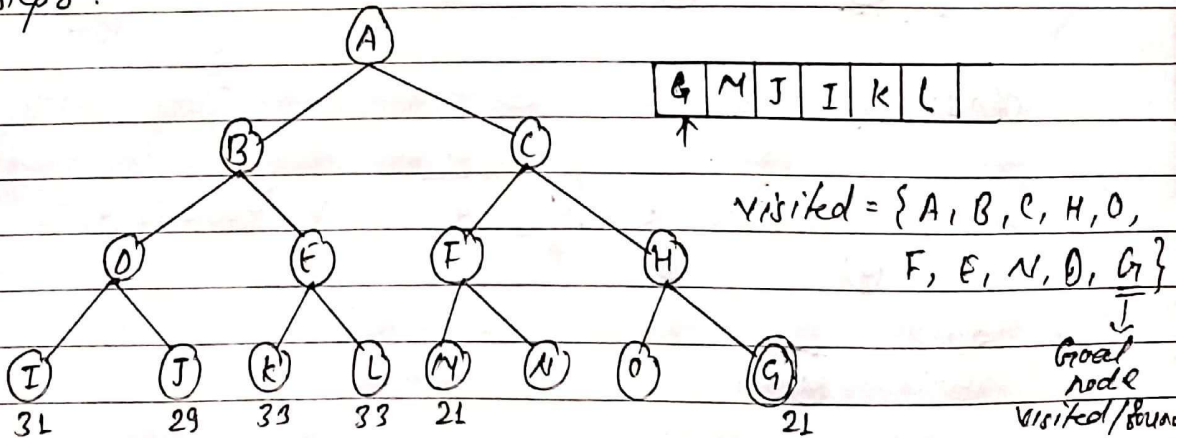
step 6:



step 7:

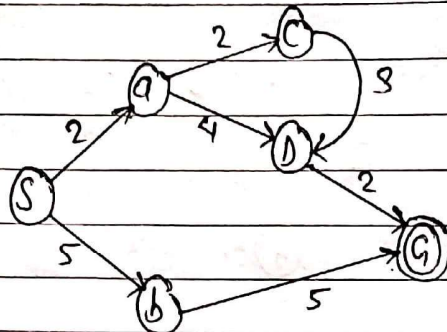


step 8:



074

Q. Given following state-space, use uniform cost search algorithm to find the goal. show each of iterations.



Here s is start state and G is goal state.

Q1

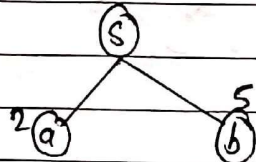
Step 1:

S^0

Queue

visited = {s}

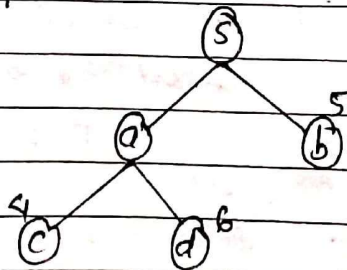
Step 2:



↑

visited = {s, a}

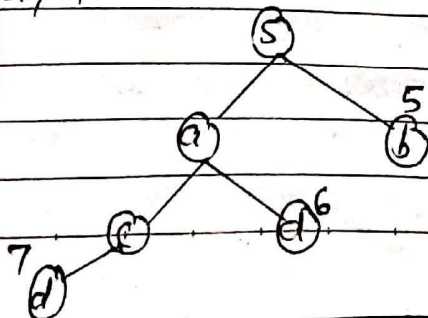
Step 3:



↑

visited = {s, a, c}

Step 4:



↑

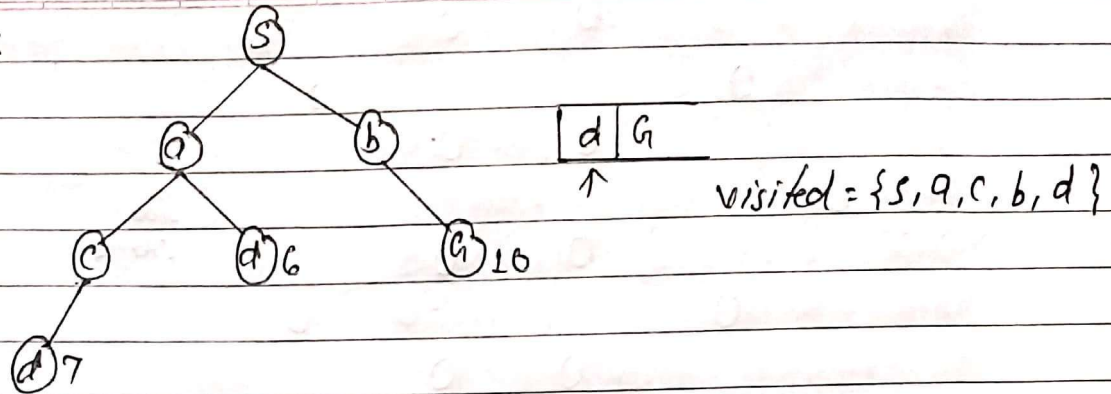
visited = {s, a, c, b}

classmate

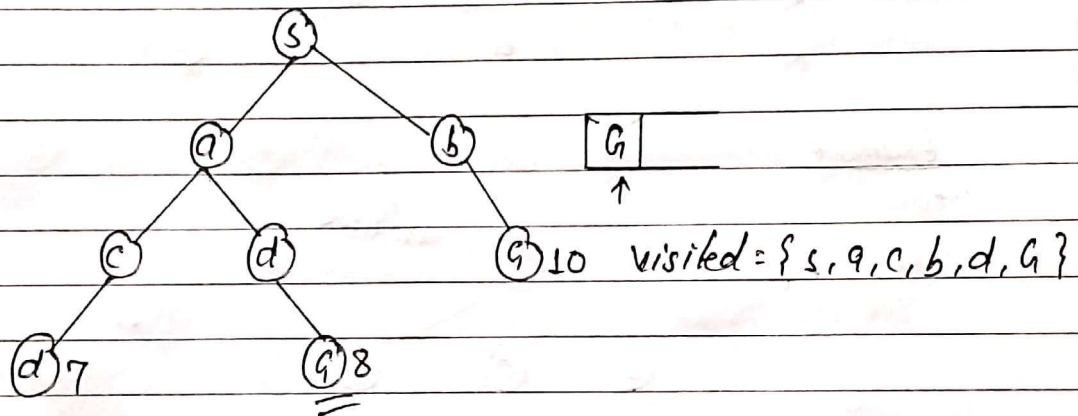
Date _____

Page _____

step 5:



step 6:

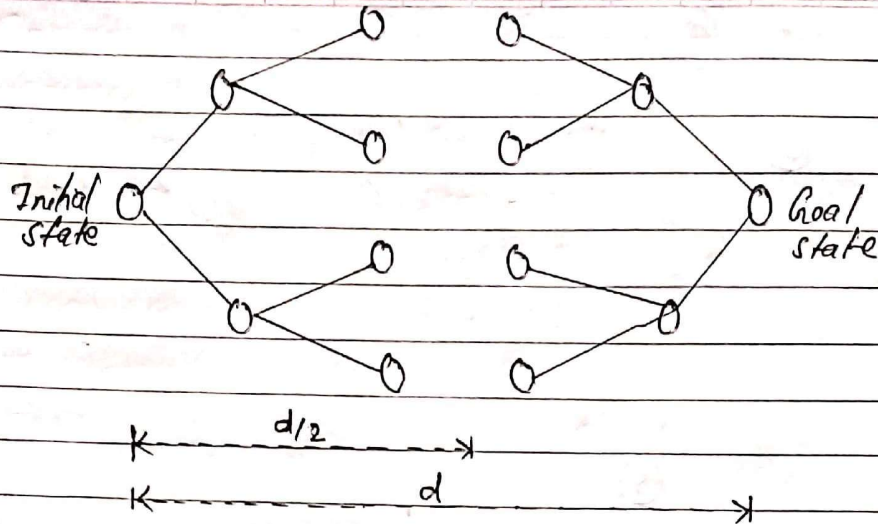


Here the value of d is the smallest but it is already visited so search goes to G having value 8. Therefore goal node found.

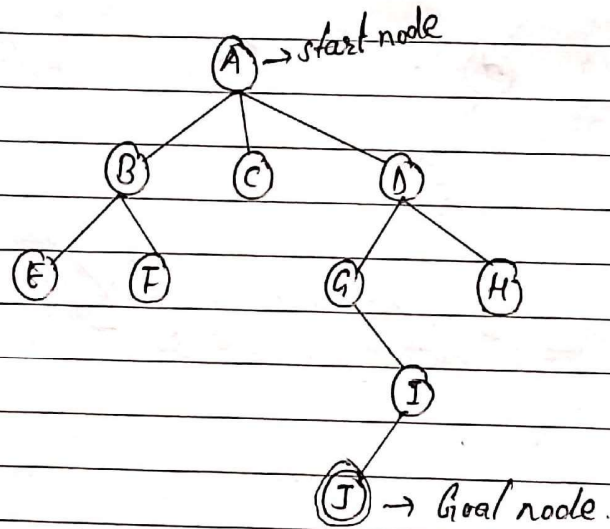
Bidirectional Search

Bidirectional search algorithm runs two simultaneous searches, one from initial state called as forward-search and other from goal node called as backward-search. The search process terminates when the searches meet a common node of the search tree.

→ Bidirectional search can use search technique such as BFS, DFS etc.



Example



Let us consider we will apply BFS from starting node and we will apply DFS from goal node.

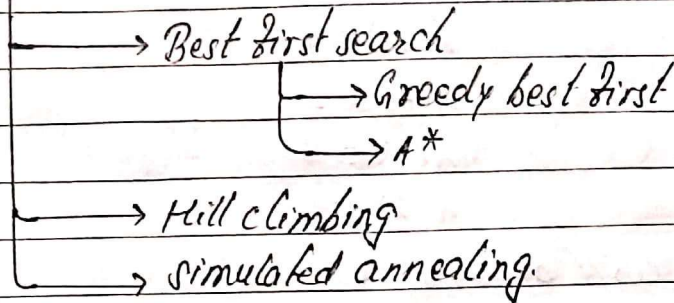
Forward-search: $A \rightarrow B \rightarrow C \rightarrow D$

Backward-search: $J \rightarrow I \rightarrow G \rightarrow D$

Here, we can see that both searching method meets a point at D. That's why we have to stop searching here and we can say that we have a path from start node to goal node. i.e.

$A \rightarrow D \rightarrow G \rightarrow I \rightarrow J$

Heuristic / Informed Search



Best first search

Nodes are expanded based on the value of evaluation function. Nodes having minimal value of evaluation function are expanded first.

1) Greedy Best first search

It expands the node that appears to be closest to the goal.

The evaluation function is defined by

$$f(n) = h(n)$$

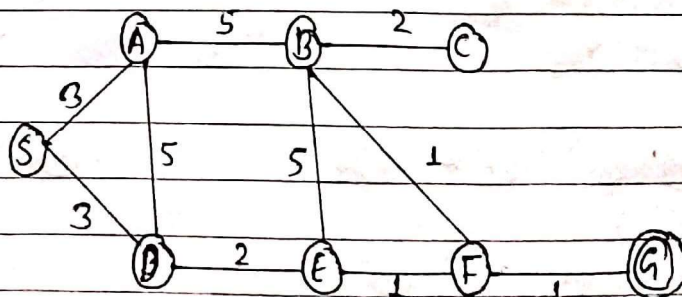
where, $h(n)$ is an estimate of cost from node n to goal/node

$h(n) = 0$ for goal node.

↳ (Heuristic value).

E.g.

①



Consider a heuristic function $h(n)$ for above state space is defined as;

$$h(S) = 10$$

$$h(E) = 4$$

$$h(D) = 7$$

$$h(F) = 2$$

$$h(A) = 6$$

$$h(C) = 3$$

$$h(B) = 4$$

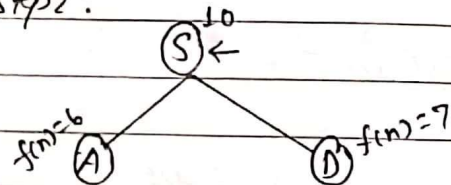
$$h(G) = 0$$

The search will be :

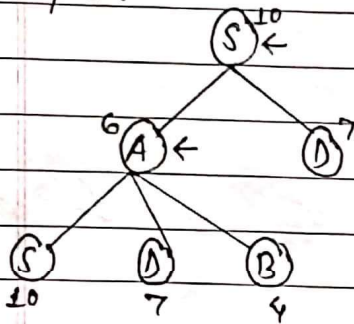
step 1:

$$(S) \quad f(n) = h(n) = 10$$

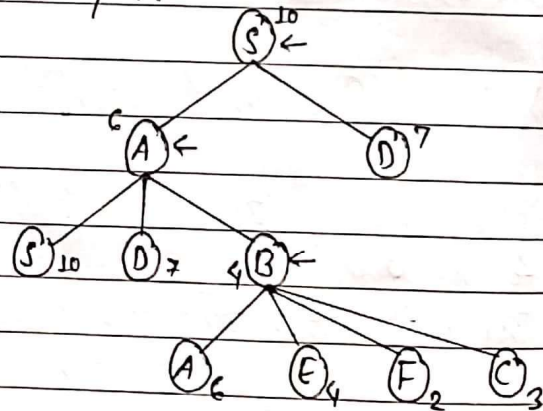
step 2:



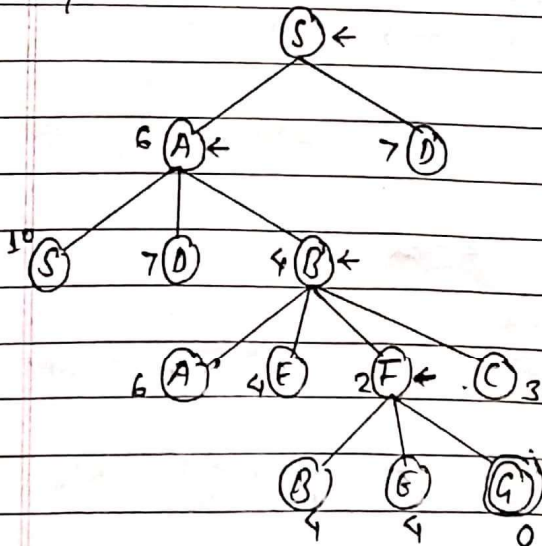
step 3:



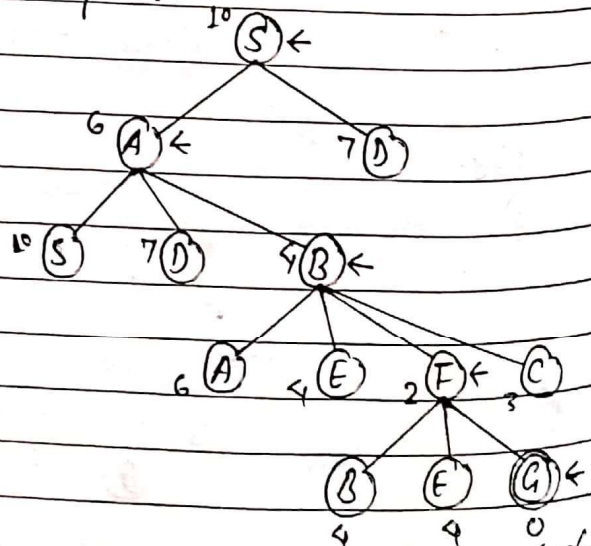
step 4:



step 5:



step 6:



classmate

Date _____

Page _____

If we change the heuristic of c as $h(c) = 2$. It sticks in loop. When search reaches at node c it sticks in loop so we can't reach at goal node.

$\therefore$ Greedy best first search is **not complete & optimal**.

{ $\rightarrow$ GBFS doesn't guarantee optimal solⁿ }
 { $\rightarrow$ It can get stuck in loop. It is not optimal }

2) **A* search**

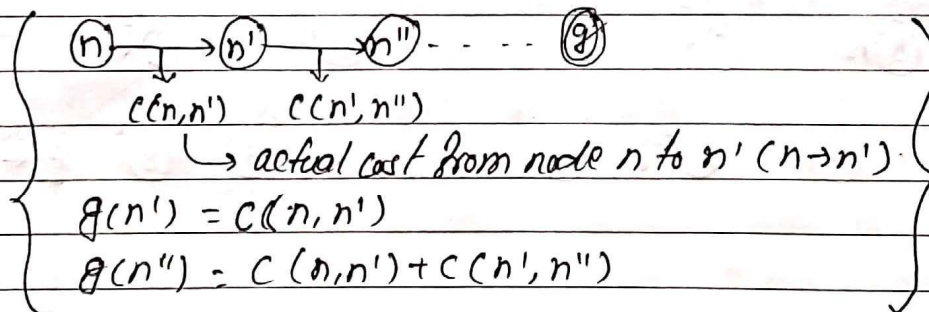
The evaluation function is defined by

$$f(n) = g(n) + h(n)$$

where, $g(n)$ = sum of actual costs incurred while travelling from root node (start node) to node n.

$h(n)$ = estimate of cost from node n to goal node.

$f(n)$ = estimated total cost of path through n to goal.

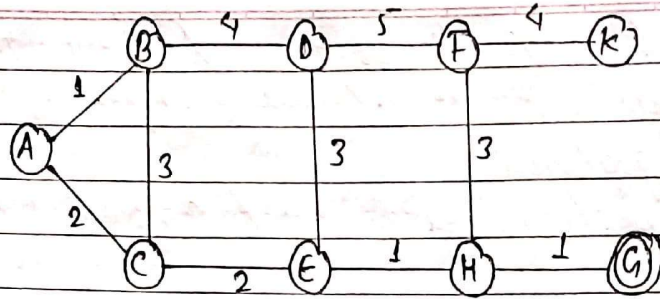


$\rightarrow$ A* search guaranteed to complete.

$\hookrightarrow$ It definitely finds the solution.

$\rightarrow$ A* search is **both complete and optimal** if an admissible heuristic is involved.

E.g.



Given $h(n)$ as

$$h(A) = 15$$

$$h(D) = 9$$

$$h(F) = 3$$

$$h(B) = 11$$

$$h(E) = 6$$

$$h(K) = 2$$

$$h(C) = 10$$

$$h(H) = 2$$

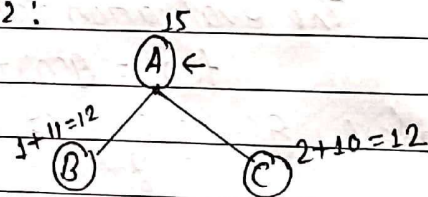
$$h(G) = 0$$

Thus the search will be,

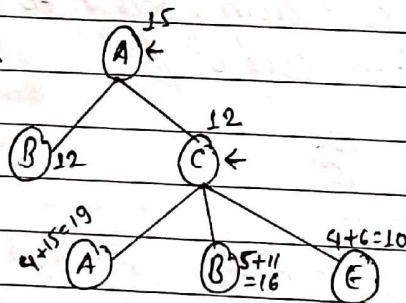
step 1:

$$A^* f(n) = g(n) + h(n) \\ = 0 + 15 = 15$$

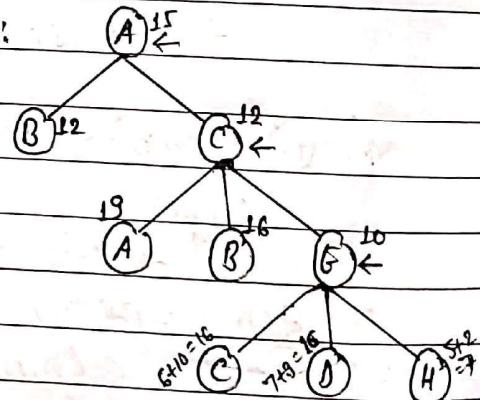
step 2:



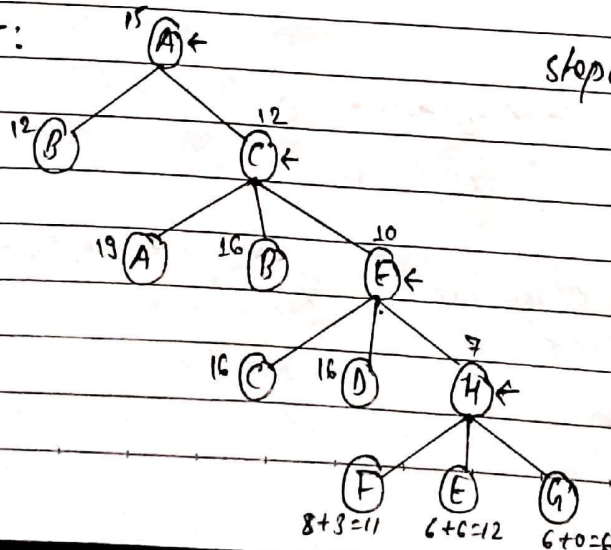
step 3:



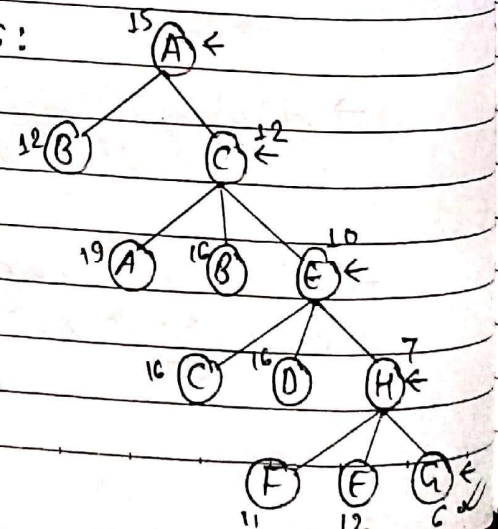
step 4:



step 5:



step 6:



classmate

Date _____

Page _____

If $h(n) = \min(\sum \frac{c(n, n')}{2})$ is given then heuristic function is calculated as

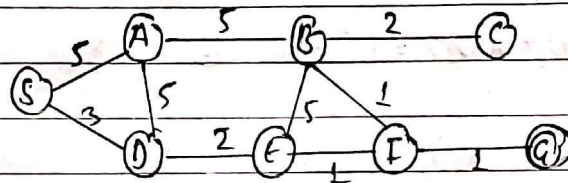
$$h(S) = \min(7/2, 14/2, 12/2, 17/2, \dots) = 7/2 = 3.5$$

$$h(A) = \min(7/2, 9/2, 12/2, 13/2, \dots) = 7/2 = 3.5$$

$$h(B) = \min(2/2, 7/2, \dots) = 2/2 = 1$$

$$h(C) = \min(4/2, 9/2, \dots) = 2$$

⋮



* Admissible Heuristic

A heuristic function is said to be admissible if for all node n , $h(n) \leq c(n)$

where, $h(n)$ is heuristic of node n to goal &

$c(n)$ is actual cost to reach goal from node n .

* Consistency (Monotonicity)

A heuristic is said to be consistent if for any node n , $h(n) \leq c(n, n') + h(n')$.

where, n' is successor of n

$h(n)$ is estimated cost from node n to goal.

$c(n, n')$ is actual cost from n to n' .

Imp.

Theorem:

A^* is optimal if $h(n)$ is admissible.

suppose G_2 is suboptimal goal.

let n be unexpanded node on shortest path to optimal goal G .

let c^* be cost of optimal goal node.

$$f(G_2) = h(G_2) + g(G_2) = g(G_2) \quad [\because h(G_2) = 0]$$

$$f(G_2) > c^* \quad \text{--- (1) since } G_2 \text{ is suboptimal.}$$

since $h(n)$ is admissible;

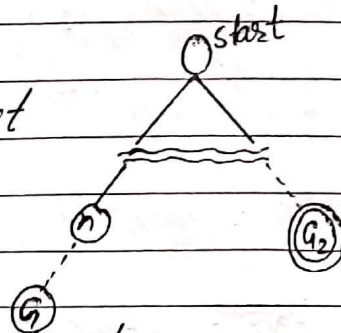
$$f(n) = g(n) + h(n) \leq c^* \quad \text{--- (2)}$$

from (1) & (2)

$$f(n) \leq c^* < f(G_2)$$

$$\therefore f(G_2) > f(n)$$

Thus A^* will never go for suboptimal goal G_2 if $h(n)$ is admissible.

**Theorem:**

If $h(n)$ is consistent, then the values of $f(n)$ along the path are non-decreasing.

Suppose n' is successor of n , then

$$g(n') = g(n) + c(n, a, n')$$

we know that,

$$f(n') = g(n') + h(n')$$

$$f(n') = g(n) + c(n, a, n') + h(n') \quad \text{--- (1)}$$

A heuristic h is consistent if

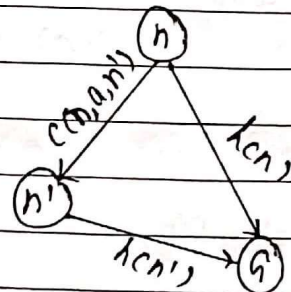
$$h(n) \leq c(n, a, n') + h(n') \quad \text{--- (2)}$$

from (1) & (2)

$$f(n') = g(n) + c(n, a, n') + h(n') \geq g(n) + h(n) = f(n)$$

$$f(n') \geq f(n)$$

$f(n)$ is non-decreasing along any path.



classmate

Date _____

Page _____

Hill climbing search

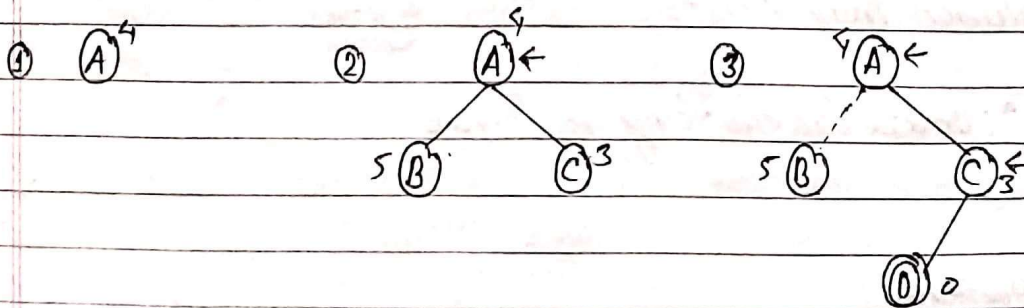
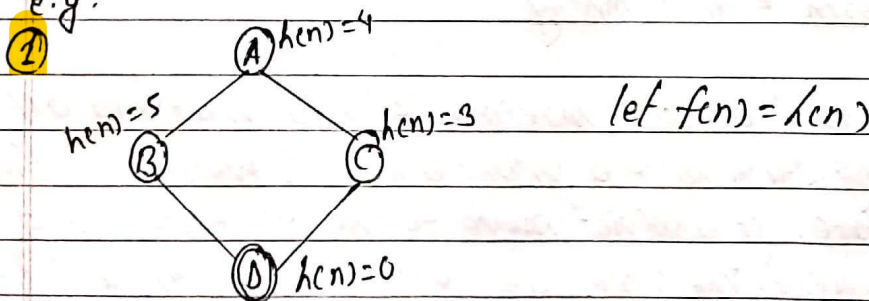
In hill climbing the basic idea is to always head towards a state which is better than the current state.

→ Hill climbing search is local search because it just try to evaluate only current node's successor without thinking ahead about where to go next.

Algorithm

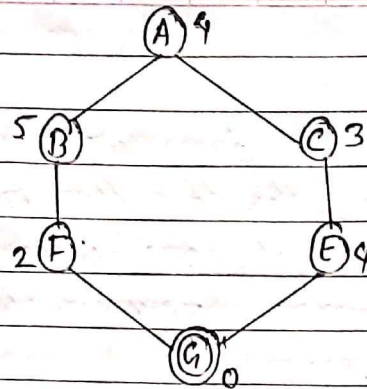
1. set current_state = initial-state
2. Until current_state = goal-state or there is no change in current state do :
 - i) Get the successors of the current state and use the evaluation function to assign a score to each successor.
 - ii) If one of the successors has a better score than the current state then make it new current state.

E.g.

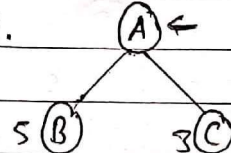


→ Hill climbing search is not complete.

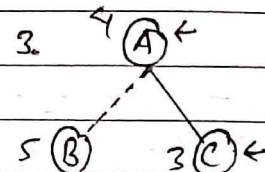
②

1. A⁴

2. A ←



3.

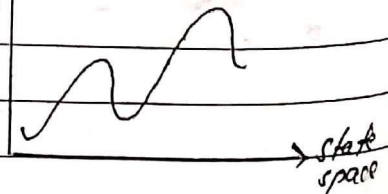


Here, the search will stop at node c. Node c's successor has no better score than c so it will stop here.
 $\therefore$ It is not complete.

Problems with Hill climbing

1. It gets stuck at local maxima. At local maxima all neighboring states have a value which is worse than the current state. It will not move to the worse state and terminate itself. The process will end even though a better solution may exist.

solⁿ: utilize backtracking technique

objective
function

2. Plateau:

An area of the search space where evaluation function is flat. On plateau all neighbors have same value.

o.f



classmate

Date _____

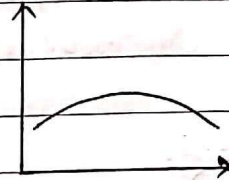
Page _____

Hence it is not possible to select the best direction.

Solⁿ: Make a big jump.

3. Ridge:

where there are steep slopes and any point on the ridge can look like ~~peak~~ peak because movement in all direction is downward. Hence the algorithm stops when it reaches this state.



Solⁿ: Move in several direction at once.

Simulated Annealing (SA)

→ allows downward steps. (Compared to hill climbing search)

→ a move is selected at random and then decides whether to accept it. If the move is better than its current position then simulated annealing will always take it. If the move is worse then it will be accepted based on some probability. The probability of accepting a worse state is given by the eqⁿ

$$p = e^{-C/t} > r$$

where,

C = change in the evaluation function

t = current value

r = a random number betⁿ 0 & 1.

Q: What is local maxima problem in hill climbing? How simulated annealing handles the problem of local maxima in hill climbing search?

classmate

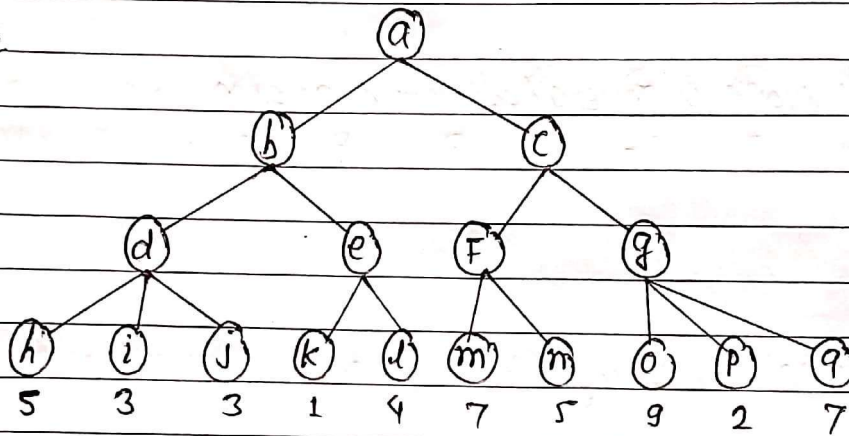
Date

Page

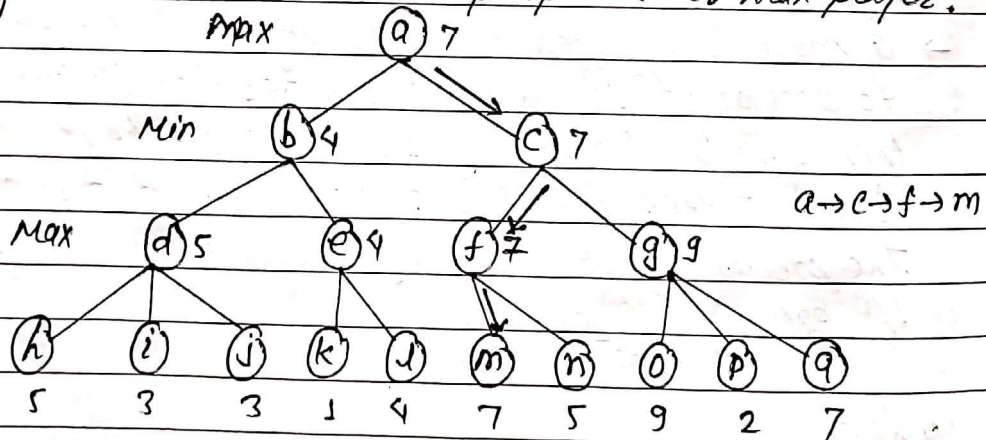
Mini-Max Search

- has two agents min & max.
- max player maximizes its utility value. Thus choose successor with highest utility.
- min player minimizes its utility value. Thus choose a successor with lowest utility.

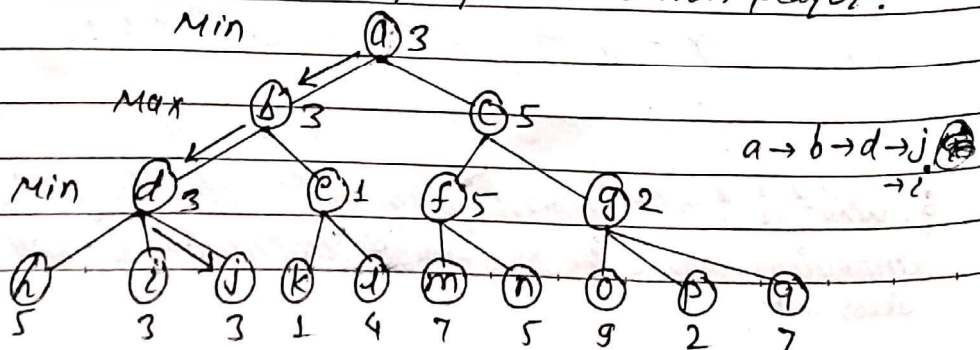
e.g.



using mini-max search from perspective of max player:



using mini-max search from perspective of min player:



classmate

Date

Page

Alpha-Beta Pruning

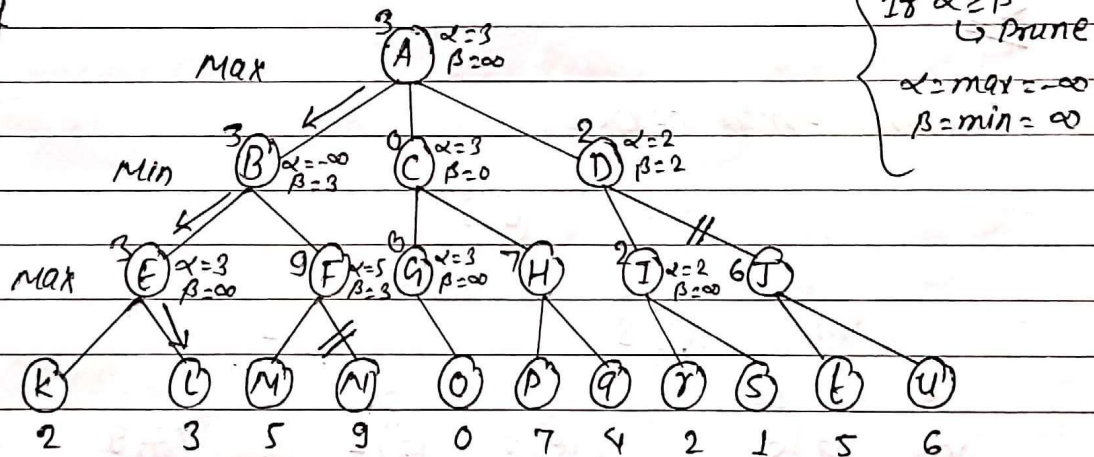
→ is a technique of eliminate unnecessary nodes from state space.

→ has two values alpha and beta.

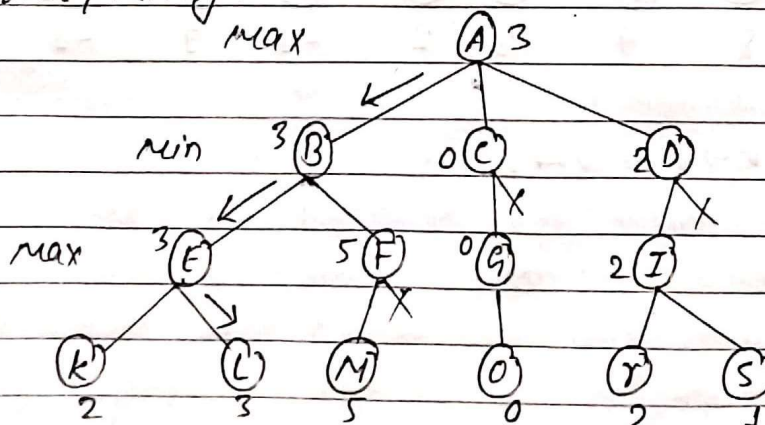
Alpha → is best (i.e. max^m) value found so far at any choice point along the path for MAX.

Beta → is best (i.e. min^m) value found so far at any choice point along the path for MIN.

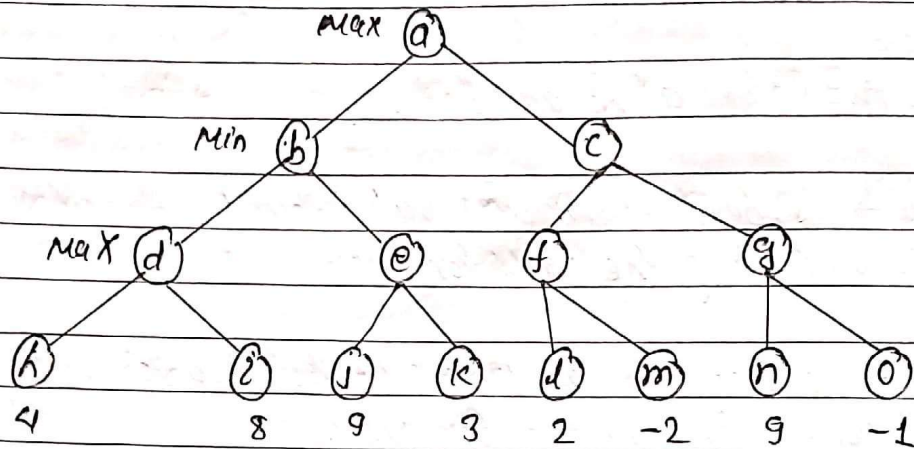
E.g



After pruning:

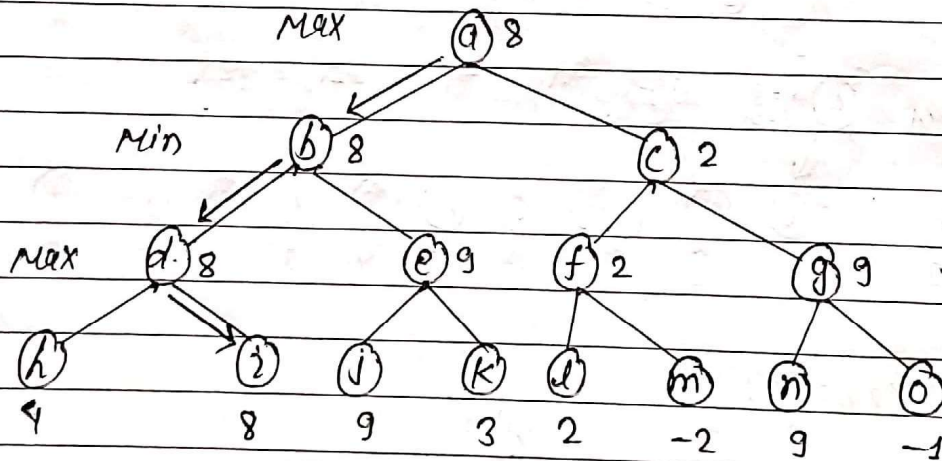


Q. Given following search space with utility, perform mini-max search and identify alpha-beta cutoff if any.



Solⁿ

The mini-max search:



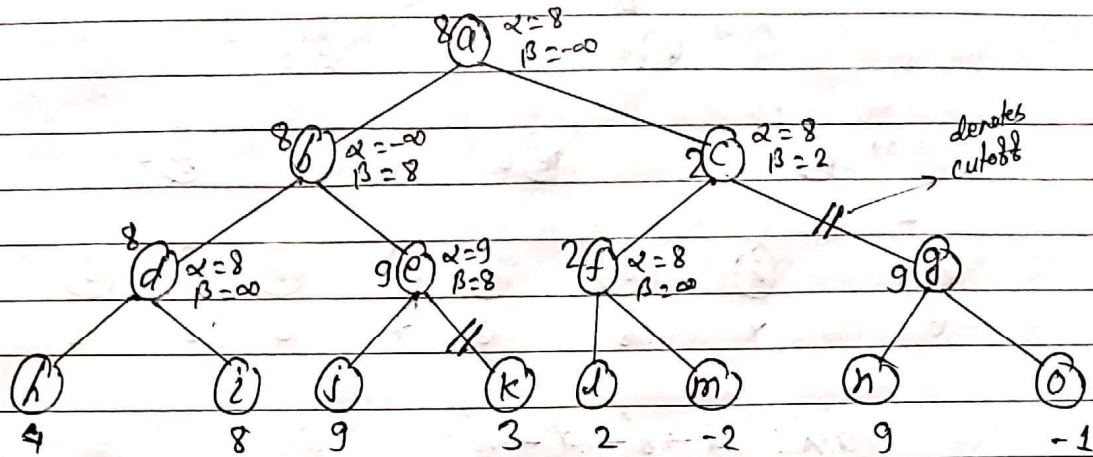
path: $a \rightarrow b \rightarrow d \rightarrow i$

classmate

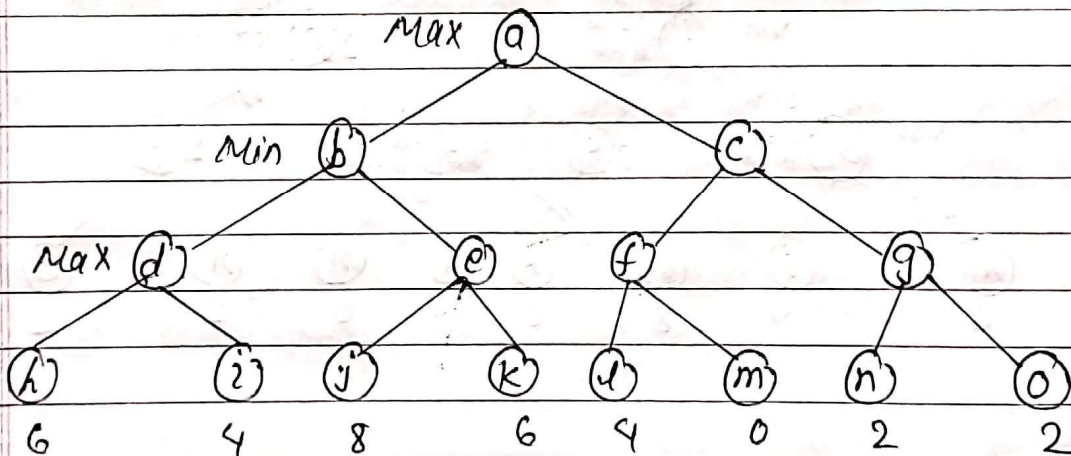
Date _____

Page _____

Alpha-beta cutoff:

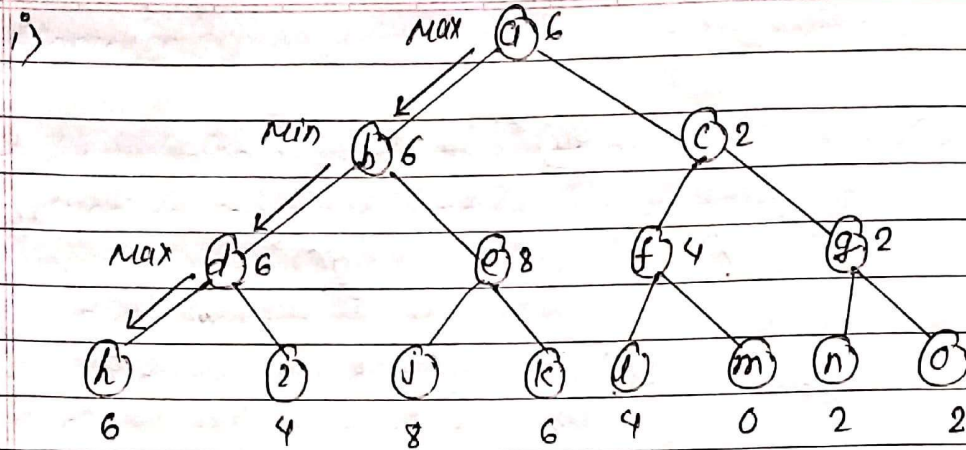


Q. Consider the following game tree

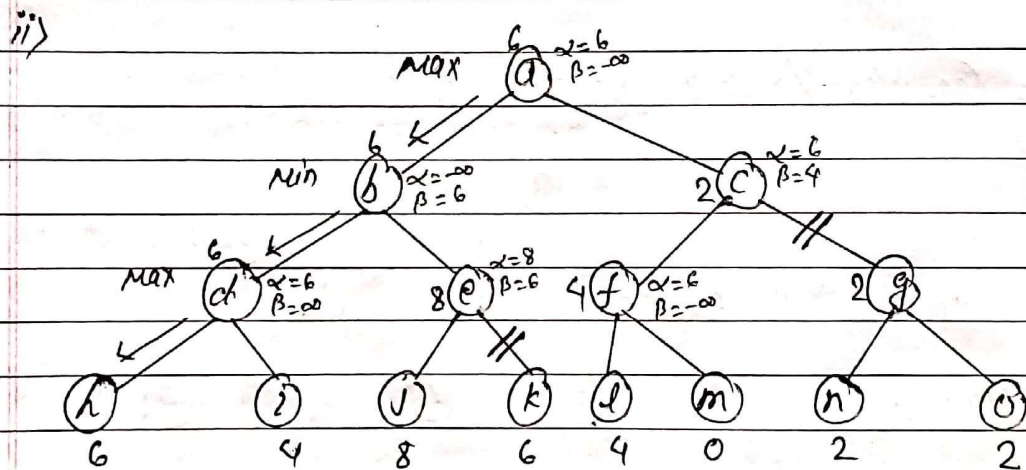


- i) use the mini-max procedures and show what moves should be chosen by the two players.
- ii) use the alpha-beta pruning procedure and show what nodes would not need to be examined.

Solⁿ



Path: $a \rightarrow b \rightarrow d \rightarrow h$



Constraint Satisfaction Problem (CSP)

CSP are representation where the problems are formulated as constraints between a set of variables. A constraint satisfaction problem (CSP) consists of:

- a set of variables $\{x_1, x_2, \dots, x_n\}$
- a finite set of domain D_i associated with each variables.
- a set of constraints $\{C_1, C_2, \dots, C_n\}$

A solution is an assignment of a value in D_i to each variable x_i such that every constraints are satisfied.

- variables represent the entities in the CSP.
- The domain of variable is a set of possible values that can be assigned to the variable.
- constraint is a set of relations between values (domain) of variables.

E.g.

1) N-Queen Problem

N-queen is a problem where given N -by- N board, you want to place N queens such that no two queens are in the same row, column or diagonal.

CSP definition:

variables : The row assignment of N queens

Domains : 1 to N

Constraints : No two queens can be on the same row, column or diagonal.

2) Map coloring

Map coloring is a problem where given a map and a set of colors, you color the regions of the map such that no adjacent regions have the same color.

CSP definition:

variables : The color of each region of the map.

Domains : The list of colors.

Constraints : No two adjacent regions can be colored the same color.